

Polygon crawling: feature edge extraction from a general polygonal surface for mesh generation

Soji Yamakawa · Kenji Shimada

Received: 23 November 2005 / Accepted: 11 May 2007 / Published online: 1 December 2009
© Springer-Verlag London Limited 2009

Abstract This article describes a method for extracting feature edges of a polygonal surface for mesh generation. This method can extract feature edges from a polygonal surface typically created by a CAD facet generator in which typical feature edge extraction methods fail due to severe nonuniformity and anisotropy. The method is based on the technique called “polygon crawling,” which samples a sequence of points on the polygonal surface by moving a point along the polygonal surface. Extracting appropriate feature edges is important for creating a coarse mesh without yielding self-intersections. Extensive tests have been performed with various CAD-generated facet models, and this technique has shown good performance in extracting feature edges.

Keywords Feature edge · Polygonal surface · Facet · Curvature estimation

1 Introduction

This article describes a method for estimating curvature and extracting feature edges of a polygonal surface for mesh generation. The proposed method takes a polygonal surface as input and extracts feature edges from the polygonal surface. The input is not limited to a dense uniform triangular mesh. Feature edge extraction methods

have been developed for a dense triangular mesh as shown in Fig. 1a, which is typically created by a laser range scanner, and a uniform mesh as shown in Fig. 1b. Some of those methods are reviewed in Sect. 2. The proposed method targets a general polygonal surface that includes concave and high aspect-ratio polygons as shown in Fig. 1c. The proposed method does not require the input polygonal surface to be uniform or dense. It assumes, however, that the input polygonal surface represents the intended geometry without as much noise as a typical scanner-generated polygonal surface would have. This method is particularly advantageous when a set of feature edges must be extracted from a triangular facet generated by a CAD facet generator, which includes triangles with high aspect ratio. Existing feature edge extraction schemes do not perform well on triangular facets that include high aspect-ratio triangles. It is also useful for extracting feature edges from a meshed model. This method also tolerates certain unevenness created by a CAD facet generator. A CAD facet generator often yields unevenness that is not included in the original CAD model due to a bug in the facet generator, a bad set of parameters given to the facet generator, or a deficiency of the original CAD model, as shown in Fig. 2. If a cylindrical surface is tessellated appropriately as shown in Fig. 2a, the normal vectors of the polygons change monotonically clockwise or counter clockwise along the circumferential direction. However, a CAD facet generator often tessellates a cylindrical surface as shown in Fig. 2b, in which case the tessellated surface includes a virtual bump, and the normal vectors no longer change monotonically along the circumferential direction.

An industrial product is often designed by a team of multiple companies with several different CAD packages. For running an analysis, all CAD models need to be converted to a common data format that can be recognized by

S. Yamakawa (✉) · K. Shimada
The Department of Mechanical Engineering,
Carnegie Mellon University, 5000 Forbes Ave.,
Pittsburgh, PA 15213, USA
e-mail: soji@andrew.cmu.edu

K. Shimada
e-mail: shimada@cmu.edu

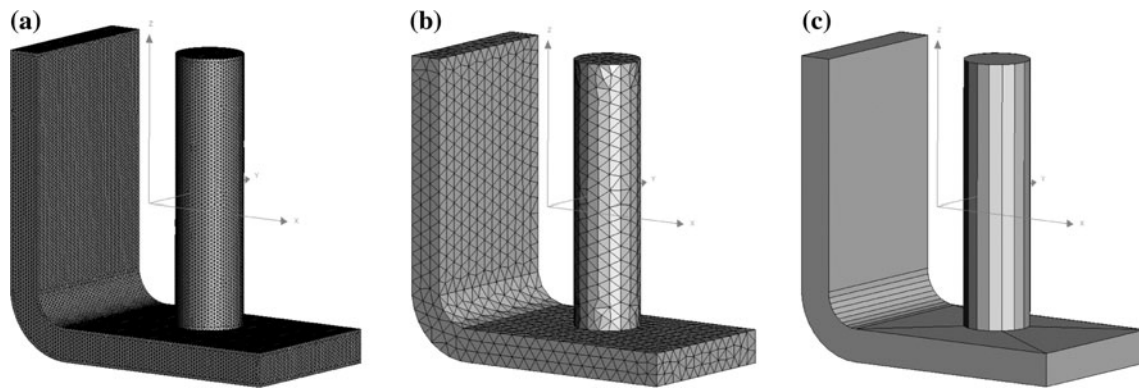


Fig. 1 Typical polygonal surfaces. **a** A dense triangular mesh typically created by a laser range scanner. **b** A good quality uniform mesh. **c** A general polygonal surface including concave and high aspect-ratio polygons, which the proposed method deals with

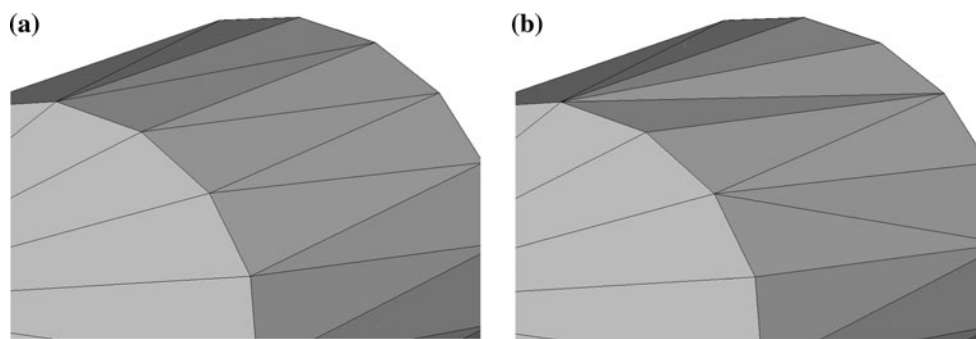


Fig. 2 Inappropriate tessellation that can be created by a CAD facet generator. **a** Appropriate tessellation of a cylindrical surface. **b** Inappropriate tessellation of a cylindrical surface

a mesh generator. However, the conversion from each CAD's native format to a common format often breaks the geometric data when the common format does not have a compatible geometry expression to an expression of the native format. In such situations, STL format, the simplest tessellated format, may be the only common format that can be exported from all the CAD packages, and feature edges need to be extracted from an STL file. This feature edge extraction from an STL file is the goal of the proposed method.

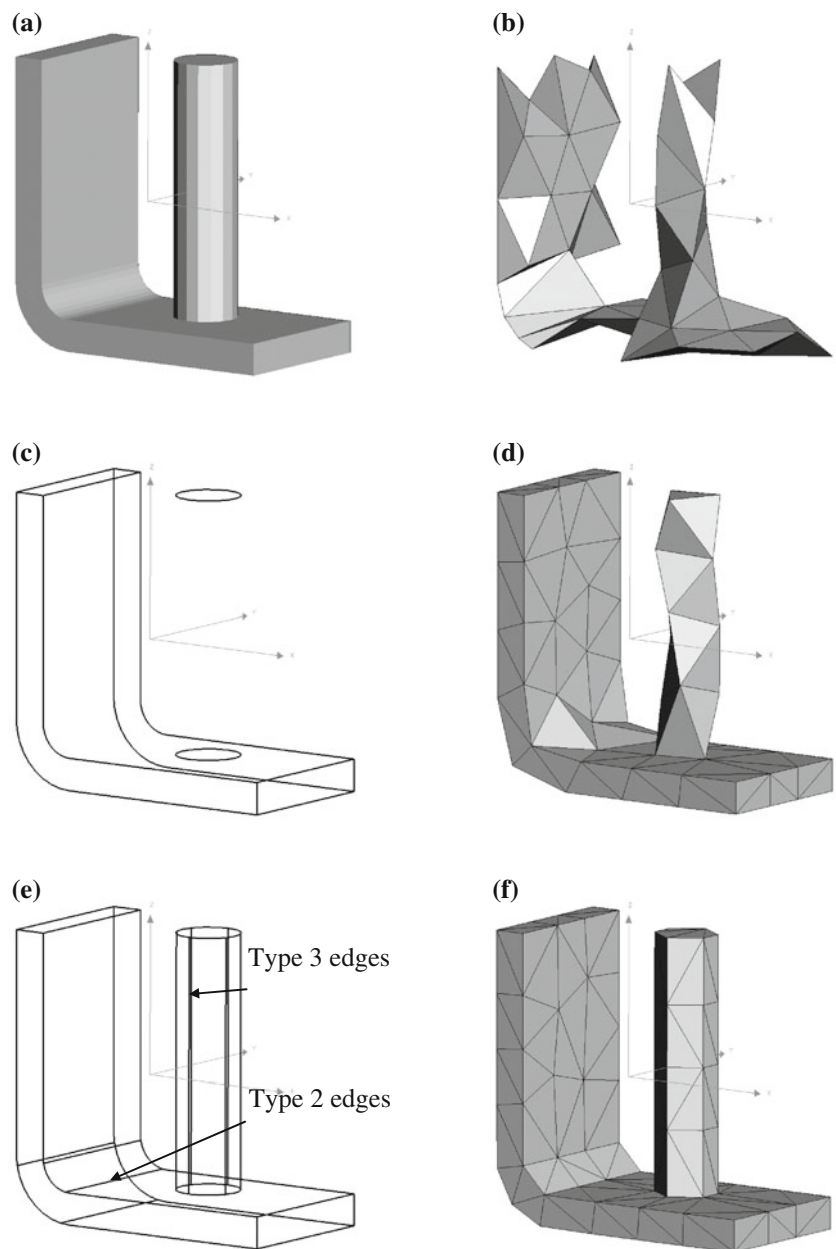
In this article, feature edges are defined as a set of edges that must be preserved, or constrained, to obtain a valid mesh. A mesh is valid if no element intersects with another, and if every edge is shared by two elements—except along the boundary of the open polygonal surface. This also applies to a mesh of a nonmanifold surface, which consists of multiple surfaces connected together with some boundaries shared by more than two surfaces. Edges along such boundaries can be shared by more than two elements. Every edge apart from the boundaries needs to be shared by two elements. The proposed method focuses on extracting the following edges:

- Type 1. Edges along sharp corners.
- Type 2. An edge in which the surface curvature measured in the direction perpendicular to the edge changes significantly across the edge.
- Type 3. Edges on the cylindrical surfaces.

Figure 3 demonstrates the effects of feature edges on mesh generation. Figure 3a shows the input model. Figure 3b shows a meshed model without constraining edges at all. The mesh clearly does not describe the input model well. Figure 3c shows edges that pertain to type 1, and Fig. 3d shows a meshed model with type 1 edges constrained. Such edges can easily be found by checking the dihedral angle of the polygons sharing an edge. However, it does not describe the fillet and the cylindrical surfaces of the input model well. Figure 3e shows edges of all three types extracted by the proposed method, and Fig. 3f shows a meshed model with constraining edges of all three types; among the three meshed examples it best describes the original geometry.

A simple method can detect only feature edges that pertain to type 1. If only type 1 edges are constrained,

Fig. 3 Effect of feature edges on mesh generation. **a** Input model, **b** meshed without constraining feature edges, **c** type 1 edges of the input model, **d** meshed with only type 1 edges constrained, **e** feature edges extracted by the proposed method, **f** meshed with feature edges constrained



sufficiently small elements must be used for preserving the original geometric features, and creating such small elements often yields an excessively dense mesh, especially when the element size needs to be small for preserving a small feature attached to a large body. However, it is more difficult to detect edges that pertain to types 2 and 3. The goal of the proposed method is to detect all three types of feature edges. The proposed method can extract edges along the boundaries of any surface as type 2 edges—a plane, a cylindrical surface, a spherical surface, a saddle surface, etc., because along such edges a sudden change of curvature occurs frequently. However, a closed (or nearly closed) cylindrical surface does not include such sudden change of curvature on the surface.

The proposed method seeks out such cylindrical surfaces and extracts edges on these surfaces as type 3 edges.

Extracting feature edges from a general polygonal surface is a much more challenging problem than extracting feature edges from a CAD model with smooth surfaces from which exact curvature at any point on the surface can be calculated. However, a polygonal surface itself does not have such information, and curvature on the surface can only be estimated. Feature edges thus can be more accurately extracted from a CAD model than from a polygonal surface. However, we often have only access to a polygonal surface for variety of reasons including incompatibility of a CAD data file between systems, confidentiality requirements of the contract, or the loss of the original

CAD model caused by a malfunction of a computer or operator error. In such cases it is necessary to develop a method for estimating curvature and extracting feature edges from a polygonal surface.

The proposed method uses a procedure called polygon crawling for estimating curvature and finding feature edges from the input polygonal surface. The polygon crawling method samples sequence of points, called a “crawling path,” on the polygonal surface starting from an initial point on the surface toward an initial crawling direction, which is perpendicular to the normal of the polygon on which the initial point is located. The curvature of the polygonal surface at the initial point along the initial direction is estimated by analyzing the crawling path. The method locates the edges where the curvature changes significantly from one side of the edge to the other by estimating and comparing curvature in the direction perpendicular to an edge on both sides of the edge. The cylindrical surface in the polygonal surface can be found by (1) finding a principal curvature direction at a point on the polygonal surface by numerically searching the direction that makes the magnitude of the estimated curvature maximum, and (2) fitting a circle to the crawling path starting from the point toward the principal curvature direction.

The organization of the article is as follows. Section 2 reviews previous work of the feature edge extraction. Section 3 explains the polygon crawling method. Section 4 explains the parameters that control the results of the proposed method. Section 5 describes how the curvature is estimated using the polygon crawling method, and Sect. 6 presents the algorithm of extracting feature edges based on the information obtained by the polygon crawling method. Section 7 shows some results, and Sect. 8 concludes the article.

2 Related work

Mangan and Whitaker [1] present a method for partitioning a dense 3D mesh. Their method calculates a curvature (or some other height function) at each vertex, and merges regions based on the curvature (or the height). Feature edges can be extracted by taking edges between partitions. They also mention the possibility of extending the method for a coarse mesh. Their method, however, is not designed for a mesh that includes high aspect-ratio triangles.

Nomura and Hamada [2] present a method for extracting feature edges from a triangular mesh. Their method first calculates a measurement called “*concavity–convexity*,” and extracts edges between a convex triangle and a concave triangle. The noise of the feature edges will be reduced by applying a thinning algorithm. The method also assumes a dense uniform mesh as input.

Jiao and Heath [3, 4] present a method for extracting feature edges. Their method finds a set of edges that the dihedral angle made by the two polygon sharing the edge exceeds the given angle threshold called θ -strong edges, and takes as a feature edge a θ -strong edge that is likely to form a smooth curve when it is connected to a neighboring θ -strong edge. Their method is designed for a uniformly sized mesh and thus not applicable to a general polygonal surface.

Sun et al. [5] present a method for detecting feature edges. This method computes the likelihood of an edge to be a feature edge, called the “edge strength,” by taking vote of tensors calculated from the normal vectors of the vertices weighted by the geodesic distance from the edge. Their method is devised for segmenting a dense uniform mesh, created typically using a laser range scanner. The method is thus not applicable to a polygonal surface created by a CAD facet generator.

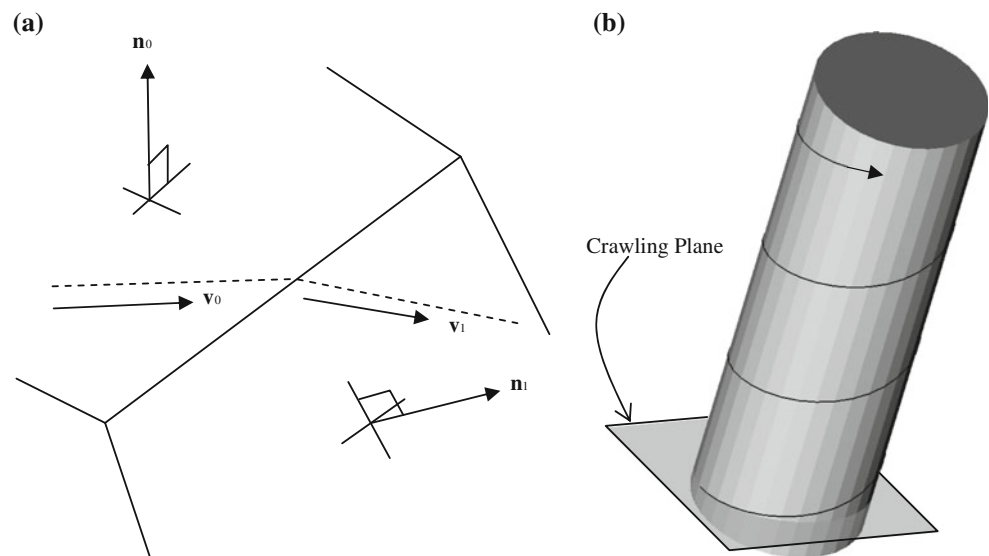
Baker presents a method for identifying feature edges from a triangular mesh [6]. His method uses two angle thresholds: one threshold identifies dominant edges, and the other threshold identifies sub-dominant edges. Dominant edges are automatically extracted as feature edges, and some sub-dominant edges are extracted based on the connection to the dominant edges. The method assumes a sufficient degree of regularity, and cannot be applied to a general polygonal surface unless some modification to the method is made.

There has also been some researches on extracting feature edges from point clouds [7, 8]. These methods extract feature edges from dense sample points. Hence, such methods can also be used for extracting edges from a dense mesh. However, these methods are not designed for taking a general polygonal surface; therefore the goal of the proposed method differs from the goals of these methods.

Jiao [9] has presented a feature edge extraction method that can tolerate certain nonuniformity and stretched polygons. However, the method cannot capture fillet boundaries, which are critical for certain applications such as finite-element mesh generation.

In summary, the existing feature edge extraction techniques for a polygonal surface can roughly be classified into two groups: (1) the techniques for a dense polygonal surface such as data created by a laser range scanner or a CT/MRI scanner, and (2) the techniques for a reasonable-quality triangular mesh. The type 1 techniques deal with substantial sampling noise, and typically extract feature edges as a band of vertices. The type 2 techniques deal with a relatively coarse mesh compared to the scanned data, and typically assumes uniformity. Neither type can deal effectively with a general polygonal surface that includes polygons with more than four vertices, concave polygons, and high aspect-ratio polygons.

Fig. 4 Crawling path on a polygonal surface. **a** Transition of the direction of motion across a boundary of a polygon, **b** a crawling path on a discretized cylindrical surface



It was also found that some edges on a surface made by an extrusion must be extracted as a feature edge for mesh generation. If the cross-section curve of the extrusion includes considerable nonuniformity of curvature or sharp corners, feature edges on such surfaces can be captured by finding a sudden change of curvature or a large dihedral angle. However, when the cross-section is a circle (when the surface is a cylinder) there is no such change of curvature or large dihedral angle. Hence, cylindrical surfaces must be detected separately and feature edges must be extracted on such surfaces. However, none of the existing methods includes cylinder detection and feature edge extraction on cylindrical surfaces.

The goal of the proposed method is to extract feature edges from a general polygonal surface, mainly a facet created by a CAD facet generator. The method tolerates a certain amount of unevenness imposed by a CAD facet generator. It also finds cylindrical surfaces and extracts feature edges on those cylindrical surfaces.

3 Polygon crawling

The proposed feature edge detection technique relies on the technique called polygon crawling, which samples sequence of points along a path on the polygonal surface called a crawling path. A crawling path is analog to a geodesic path of a smooth surface. However, a crawling path is a set of line segments as opposed to a smooth geodesic path of a smooth surface.

A crawling path is defined by an initial point on a polygonal surface, an initial direction, and termination criteria. An initial direction must be perpendicular to the normal of the polygonal surface at the initial point. If the

initial point is on an edge or a vertex of the polygonal surface, the initial direction can be a normal of one of the polygons sharing the edge or the vertex. Termination criteria can be anything; typically it is based on the length and/or the total angular change of the direction. For curvature estimation, the initial point is the point where the curvature is estimated, and the initial direction is the direction in which the curvature is estimated. The termination criteria for estimating curvature are discussed in Sect. 5.2.

Assume that a sample point is located at the given initial point and moving toward the initial direction. The sample point maintains the same direction of motion while it is inside the polygon until it reaches boundary—an edge or a vertex—and moves out of the polygon. When the sample point moving toward $\mathbf{v}_0$ direction hits the boundary and moves out of the polygon whose unit normal is $\mathbf{n}_0$, and it moves into a polygon whose unit normal is $\mathbf{n}_1$, the direction of motion is changed to $\mathbf{v}_1 = \mathbf{R}(\mathbf{n}_0 \rightarrow \mathbf{n}_1)\mathbf{v}_0$, where $\mathbf{R}(\mathbf{n}_0 \rightarrow \mathbf{n}_1)$ is a 3×3 affine transformation matrix that rotates $\mathbf{n}_0$ into $\mathbf{n}_1$ about $\mathbf{n}_0 \times \mathbf{n}_1$ as shown in Fig. 4a. The sample point keeps moving until the termination criteria is met. A crawling path is then written as $C = \{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_N\}$, where $\mathbf{p}_i$ is the point at which the sample point crosses an edge or a vertex for $(i + 1)$ th time. Figure 4b shows an example of a crawling path on a discretized cylindrical surface.

We also define a “crawling plane” as $(\mathbf{x} - \mathbf{o}_{cp}) \cdot (\mathbf{v}_0 \times \mathbf{n}_0) = 0$, where $\mathbf{x}$ is a point on the crawling plane, $\mathbf{o}_{cp}$ is the initial point of the crawling path, $\mathbf{v}_0$ is a unit vector parallel to the initial direction of the crawling path, and $\mathbf{n}_0$ is the normal of the polygon on which the initial point is lying. A crawling plane is shown in Fig. 4b by the semi-transparent rectangle.

Table 1 Input parameters

ω_{shallow}	If two vectors make an angle less than ω_{shallow} , two vectors are considered to be equivalent.
ω_{sharp}	If normal vectors of two polygons sharing an edge make more than ω_{sharp} , the edge is considered to be a sharp corner. It is also used for termination condition of the crawling method detailed in Sect. 5.2.
ω_{twist}	An angle threshold used for a termination condition of the crawling method detailed in Sect. 5.2.
γ_1	If the estimated curvatures in the direction perpendicular to an edge changes more than γ_1 times across the edge, and the two curvatures estimations are reliable, the edge is extracted as a feature edge, detailed in Sect. 6.2.
γ_2	If the estimated curvatures in the direction perpendicular to an edge changes more than γ_2 times across the edge, and only one of the two curvatures estimations is reliable, the edge is extracted as a feature edge, detailed in Sect. 6.2.
ω_{cyl1}	An angle threshold for finding a partial cylinder (a cylinder with the angle of arc of the cross-section less than 360°) represented by a set of polygons, detailed in Sect. 6.3.
ω_{cyl2}	An angle threshold for finding a complete cylinder (a cylinder with the angle of arc of the cross-section equal to 360°) represented by a set of polygons, detailed in Sect. 6.3.
N_{cyl}	The required number of subdivision around a complete cylinder, detailed in Sect. 6.3.

As can be seen from Fig. 4b, a crawling path can move away from the crawling plane. We define a “twist angle” at each point on a crawling path for measuring a directional deviation of the crawling direction from the crawling plane. If the direction of the crawling path is $\mathbf{v}_n$ ($\mathbf{v}_n$ is a unit vector) at point $\mathbf{x}$ on the crawling path, the twist angle is defined as $\theta_{\text{tw}} = \text{asin}(|\mathbf{v}_n \cdot \mathbf{n}_{\text{cp}}|)$, where $\mathbf{n}_{\text{cp}}$ is the normal of the crawling plane written as $\mathbf{n}_{\text{cp}} = (\mathbf{v}_0 \times \mathbf{n}_0)$.

4 User-specified parameters

The result of the proposed method can be controlled by eight parameters as shown in Table 1. For obtaining the best result, these parameters must be adjusted for the meshing requirements and the behavior of the CAD facet generator. However, because it is difficult to quantify the behavior of the CAD facet generator, a good combination of these parameters can only be found empirically. The experiments performed in this research suggest that the same set of parameters works well for the polygonal surfaces created by the same CAD facet generator.

Smaller ω_{shallow} , ω_{sharp} , ω_{twist} , γ_1 , and γ_2 capture more feature edges and increase the sensitivity to noise. Parameters ω_{cyl1} and ω_{cyl2} also control the sensitivity to noise, and smaller ω_{cyl1} and ω_{cyl2} help capture cylindrical surfaces more precisely and increase the sensitivity to noise. Hence, if the CAD facet generator creates a polygonal surface with little unevenness, these parameter values can be smaller, and the method will still yield satisfactory results; or, if the CAD facet generator creates an uneven (noisy) polygonal surface, these parameter values should be increased. The value of N_{cyl} depends on how smooth a cylindrical surface must be when it is meshed. Although a larger value of N_{cyl} will yield a smoother mesh on a cylindrical surface, it may excessively increase the number of elements. A set of parameters that gave a good performance for majority of the test models is presented in Sect. 7.

5 Estimating curvature by polygon crawling

The proposed method estimates curvature on the polygonal surface by analyzing a crawling path. The goal is to estimate a curvature of the polygonal surface at sample point $\mathbf{x}$ measured in direction $\mathbf{v}_0$. In the following, a curvature estimation scheme of 2D line segments is briefly reviewed, and it is then extended for estimating curvature of a crawling path, which is essentially the curvature estimation of the polygonal surface measured in the initial crawl direction.

5.1 Curvature estimation of 2D line segments

The curvature of a curve is defined as $\kappa = 1/r_c$, where r_c is the curvature radius. Hence, two tangential vectors at the two end points of a partial circular arc of length l_{arc} makes angle θ , the curvature of the arc is calculated as $\kappa = 1/r = \theta/l_{\text{arc}}$, because $l_{\text{arc}} = r\theta$ as shown in Fig. 5a. Therefore, the curvature of a curve can be approximated by the angular change of a tangential vector divided by the arc length required for yielding the change.

Similarly, the curvature of line segments can be estimated by the angular change of the average tangential vector divided by the length required to yield the change. If $\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_N$ are the vertices of line segments, average tangential vectors are calculated as:

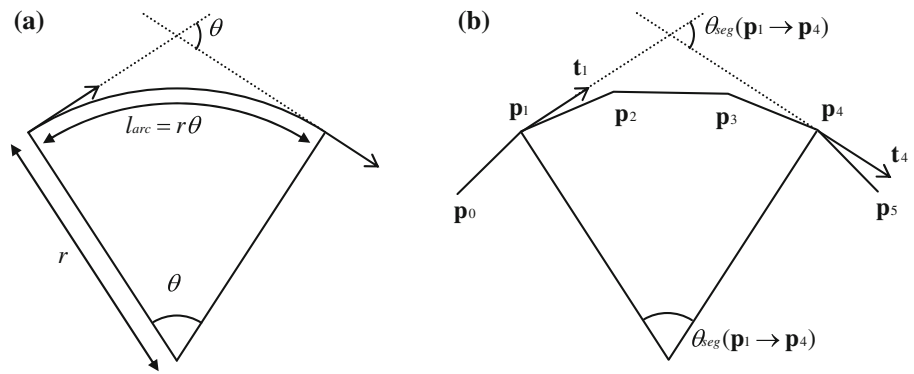
$$\begin{aligned} \mathbf{t}_0 &= (\mathbf{p}_1 - \mathbf{p}_0)/|\mathbf{p}_1 - \mathbf{p}_0| \quad \text{at } \mathbf{p}_0, \\ \mathbf{t}_N &= (\mathbf{p}_N - \mathbf{p}_{N-1})/|\mathbf{p}_N - \mathbf{p}_{N-1}| \quad \text{at } \mathbf{p}_N, \text{ and} \\ \mathbf{t}_i &= (\mathbf{p}_{i+1} - \mathbf{p}_{i-1})/|\mathbf{p}_{i+1} - \mathbf{p}_{i-1}| \quad \text{at } \mathbf{p}_i (0 < i < N). \end{aligned}$$

Curvature on line segments between $\mathbf{p}_i$ and $\mathbf{p}_j$ ($i < j$) can be estimated as

$$\kappa(\mathbf{p}_i \rightarrow \mathbf{p}_j) = \theta_{\text{seg}}(\mathbf{p}_i \rightarrow \mathbf{p}_j)/l_{\text{seg}}(\mathbf{p}_i \rightarrow \mathbf{p}_j),$$

where $\theta_{\text{seg}}(\mathbf{p}_i \rightarrow \mathbf{p}_j)$ is the angular change of the average tangential vector between $\mathbf{p}_i$ and $\mathbf{p}_j$, and $l_{\text{seg}}(\mathbf{p}_i \rightarrow \mathbf{p}_j) = \sum_{k=i}^{j-1} |\mathbf{p}_{k+1} - \mathbf{p}_k|$.

Fig. 5 Estimating curvature in 2D. **a** Calculating the curvature of an arc, **b** estimating the curvature of line segments



For example, when line segments are defined by six points, $\mathbf{p}_0, \mathbf{p}_1, \dots$, and $\mathbf{p}_5$, curvature of the line segments between $\mathbf{p}_1$ and $\mathbf{p}_4$ is estimated as

$$\kappa(\mathbf{p}_1 \rightarrow \mathbf{p}_4) = \theta_{\text{seg}}(\mathbf{p}_1 \rightarrow \mathbf{p}_4) / \left(\sum_{K=1}^3 |\mathbf{p}_{K+1} - \mathbf{p}_K| \right).$$

As can be seen from Fig. 5b, $\sum_{K=1}^3 |\mathbf{p}_{K+1} - \mathbf{p}_K|$ is analog to the arc length of Fig. 5a.

5.2 Curvature estimation of a crawling path

To extend the 2D curvature estimation scheme to a crawling path, the sign of the angular change of the average tangential vector must be taken into account because the curvature must be positive if the polygonal surface is convex along the crawling path, and negative if it is concave. Let $\mathbf{p}_0, \mathbf{p}_1, \dots$, and $\mathbf{p}_N$ be points of a crawling path, $\bar{\mathbf{p}}_i (0 \leq i \leq N)$ is the projection of $\mathbf{p}_i$ on the crawling plane, and $\bar{\mathbf{t}}_0, \bar{\mathbf{t}}_1, \dots, \bar{\mathbf{t}}_N$ are average tangential vectors calculated from $\bar{\mathbf{p}}_0, \bar{\mathbf{p}}_1, \dots, \bar{\mathbf{p}}_N$. The signed angular change of the tangential vector from $\bar{\mathbf{t}}_i$ to $\bar{\mathbf{t}}_{i+1}$ is calculated as:

$$\theta_{\text{seg}}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_{i+1}) = a$$

$$\theta_{\text{seg}}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_{i+1}) = -a$$

where $\mathbf{n}_0$ is the normal vector of the polygonal surface at the initial point of the crawling path. The angular change of the tangential vector from $\bar{\mathbf{t}}_i$ to $\bar{\mathbf{t}}_j$ ($i < j$) is calculated as:

$$\theta_{\text{seg}}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j) = \sum_{K=i}^{j-1} \theta_{\text{seg}}(\bar{\mathbf{p}}_K \rightarrow \bar{\mathbf{p}}_{K+1}).$$

The curvature of the projected crawling path between $\bar{\mathbf{p}}_i$ and $\bar{\mathbf{p}}_j$ can be estimated as

$$\kappa_{\text{seg}}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j) = \theta_{\text{seg}}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j) / l_{\text{seg}}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j).$$

For the purpose of estimating the curvature of the polygonal surface at $\mathbf{x}$ in $\mathbf{v}_0$ direction, the following two issues still need to be considered:

1. Termination criteria for sampling a crawling path, and
2. Choice of i and j .

The proposed curvature estimation method uses three user-specified parameters, sharp angle threshold— ω_{sharp} , twist angle threshold— ω_{twist} , and shallow angle threshold— ω_{shallow} for defining the termination criteria. The proposed method samples two crawling paths $C_0 = \{\mathbf{a}_0, \mathbf{a}_1, \dots, \mathbf{a}_N\}$, and $C_1 = \{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_M\}$. Projection of C_0 on the crawling plane is $\bar{C}_0 = \{\bar{\mathbf{a}}_0, \bar{\mathbf{a}}_1, \dots, \bar{\mathbf{a}}_N\}$, and C_1 $\bar{C}_1 = \{\bar{\mathbf{b}}_0, \bar{\mathbf{b}}_1, \dots, \bar{\mathbf{b}}_M\}$. The initial point of both C_0 and C_1 are $\mathbf{x}$, and the initial crawling direction of C_0 is $\mathbf{v}_0$ and of C_1 is $-\mathbf{v}_0$. For both directions, the points are sampled until:

1. total angular change of the average tangential vector on the crawling plane exceeds ω_{sharp} , i.e., both $\theta_{\text{seg}}(\bar{\mathbf{a}}_0 \rightarrow \bar{\mathbf{a}}_N)$ and $\theta_{\text{seg}}(\bar{\mathbf{b}}_0 \rightarrow \bar{\mathbf{b}}_M)$ do not exceed ω_{sharp} ,
2. angular change of the normal vectors of polygons in one step of crawling exceeds ω_{sharp} , or
3. twist angle exceeds ω_{twist} .

After creating two crawling paths, they are merged into a crawling path:

$$C_{\text{all}} = \{\mathbf{p} - (M+1), \mathbf{p} - M, \dots, \mathbf{p}_N\} \\ = \{\mathbf{b}_M, \mathbf{b}_{M-1}, \dots, \mathbf{b}_0, \mathbf{a}_0, \dots, \mathbf{a}_N\}.$$

Projection of C_{all} on the crawling plane is $\bar{C}_{\text{all}} = \{\bar{\mathbf{p}}_{-(M+1)}, \bar{\mathbf{p}}_{-M}, \dots, \bar{\mathbf{p}}_N\}$. Figure 6a shows an example of a merged crawling path.

From the merged crawling path, $\bar{C}_{\text{all}}$, $\kappa_{\text{seg}}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j)$ can be computed for any i and j that satisfy $i < j$, $-(M+1) \leq i$, $i \leq N-1$, $-M \leq j$, and $j \leq N$, and the curvature of the polygonal surface at $\mathbf{x}$ in $\mathbf{v}_0$ direction is found by choosing appropriate i and j . However, it is very difficult to automatically choose appropriate i and j . Instead of choosing only one set of i and j , the proposed method estimates the curvature from some pairs of $l_{\text{seg}}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j)$ and $\theta_{\text{seg}}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j)$, or length–angle pairs, written as $\mathbf{h}_k = (l_k, \theta_k)$, by the following algorithm.

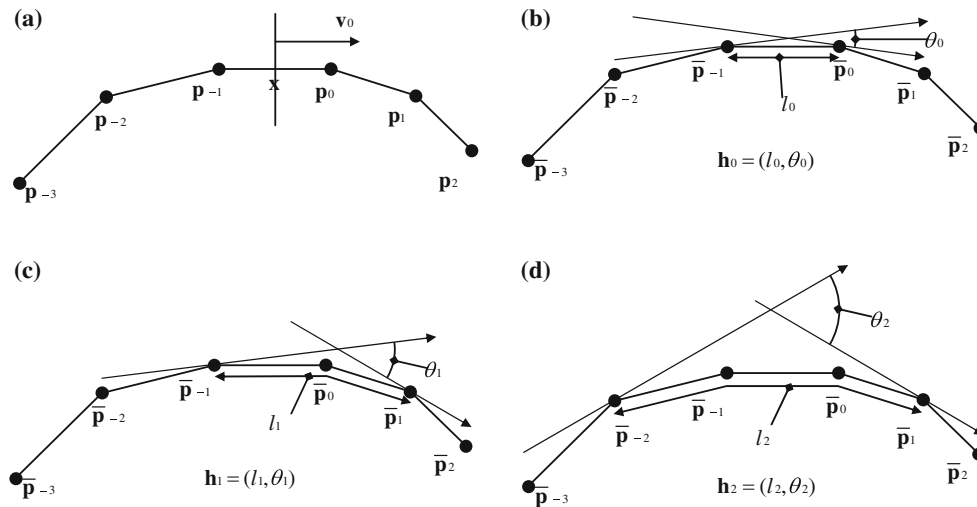


Fig. 6 Crawling path C_{all} ($M = N = 2$) and length-angle pairs. **a** Merged crawling path, **b** first length-angle pair sample ($i = -1, j = 0$), **c** second length-angle pair sample ($i = -1, j = 1$), **d** third length-angle pair sample ($i = -2, j = 1$)

Let $k = 0$
 Let i be the largest integer that satisfies:
 $\theta_{seg}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_0) \geq \omega_{shallow}$ and $i < 0$
 (If none satisfies, $i = -(M + 1)$)
 Let j be the smallest integer that satisfies:
 $\theta_{seg}(\bar{\mathbf{p}}_{-1} \rightarrow \bar{\mathbf{p}}_j) \geq \omega_{shallow}$ and $0 \leq j$
 (If none satisfies, $j = N$)
 Do:
 Set $\mathbf{h}_k = (l_k, \theta_k) = (l_{seg}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j), \theta_{seg}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j))$
 Find the next value of j , j' , which satisfies:
 $\theta_{seg}(\bar{\mathbf{p}}_j \rightarrow \bar{\mathbf{p}}_{j'}) \geq \omega_{shallow}$, $j \leq N$, and $j < j'$.
 (If none satisfies, $j' = N$)
 Find the next value of i , i' , which satisfies:
 $\theta_{seg}(\bar{\mathbf{p}}_{i'} \rightarrow \bar{\mathbf{p}}_i) \geq \omega_{shallow}$, $-(M + 1) < i$, and $i' < i$.
 (If none satisfies, $i' = -(M + 1)$)
 If $l_{seg}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_0) < l_{seg}(\bar{\mathbf{p}}_{-1} \rightarrow \bar{\mathbf{p}}_j)$
 replace i with i' ,
 Otherwise
 replace j with j' .
 Increment k .
 Until one of
 $\theta_{seg}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_j) > \omega_{sharp}$ or
 $(i = -(M + 1) \text{ and } j = N)$ is satisfied.

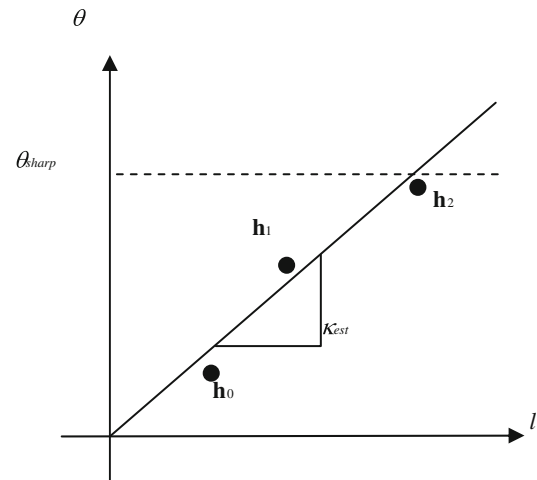


Fig. 7 Estimating the curvature by fitting a linear function to length-angle pairs

$\theta_{seg} = \kappa_{est} l_{seg}$ to $\mathbf{h}_k$ as shown in Fig. 7; and κ_{est} is written as:

$$\kappa_{est}(\mathbf{x}, \mathbf{v}_0) = \sum_k l_k \theta_k / \sum_k l_k^2,$$

which gives an estimated curvature at $\mathbf{x}$ measured in $\mathbf{v}_0$ direction.

5.3 Reliability of the curvature estimation for a crawling path

This algorithm updates i and j so that the growth of $l_{seg}(\bar{\mathbf{p}}_i \rightarrow \bar{\mathbf{p}}_0)$ and $l_{seg}(\bar{\mathbf{p}}_{-1} \rightarrow \bar{\mathbf{p}}_j)$ are balanced. Figure 6b–d shows examples of length-angle pairs.

Finally, the curvature is estimated from the length-angle plot. On length-angle plot with length as the lateral axis and angle as the vertical axis, the slope of a function $\theta = f(l)$ describes the curvature. Hence, the curvature can be estimated by the least-square fitting of function

A reliability measure of the estimated curvature must be taken into account to avoid excessive extraction of feature edges. The proposed method extracts feature edges by finding a sudden change of the estimated curvature across the edge. If the estimated curvature is subject to

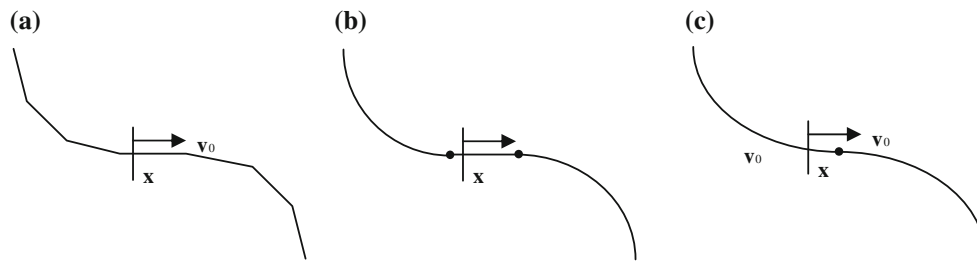


Fig. 8 Ambiguity of possible curves and lines represented by the crawling path. **a** Crawling path, **b** possible curves and straight line for the crawling path, **c** possible two curves for the crawling path

considerable error, it makes a sudden change of the estimated curvature, and the proposed method may falsely consider some edges as feature edges.

The curvature estimation scheme explained in Sects. 5.1 and 5.2 becomes inaccurate where the curvature changes from negative to positive. For example, for the crawling path shown in Fig. 8a, the exact curvature could be zero because the sample point is on a straight line connecting two curves as shown in Fig. 8b, or it could be nonzero because the sample point is on one of the two curves represented by the sampled crawling path.

The proposed method tests the reliability of the curvature estimation as follows. Let I and J be smallest i and largest j used for sampling length–angle pairs. The curvature estimation is considered to be unreliable if all three conditions of the following are satisfied:

$$\theta_{\text{seg}}(\mathbf{p}_I \rightarrow \mathbf{p}_0)\theta_{\text{seg}}(\mathbf{p}_{-1} \rightarrow \mathbf{p}_I) < 0 \quad (1)$$

$$|\theta_{\text{seg}}(\mathbf{p}_0 \rightarrow \mathbf{p}_J)| > \omega_{\text{shallow}} \quad (2)$$

$$|\theta_{\text{seg}}(\mathbf{p}_i \rightarrow \mathbf{p}_0)| > \omega_{\text{shallow}} \quad (3)$$

These conditions check whether the crawling path bends considerably to both positive and negative curvature directions. If at least one of the three conditions is not satisfied, the curvature estimation is considered to be reliable.

5.4 Finding principal curvature

The principal curvature at a point on the polygonal surface can be found by numerically maximizing the absolute value of the curvature estimated by the scheme explained in Sect. 5.2. Let $\mathbf{x}$ be a point on the polygonal surface, and $\mathbf{n}$ be the normal of the polygonal surface at $\mathbf{x}$. $\mathbf{i}$ is an arbitrary unit vector that satisfies $\mathbf{n} \cdot \mathbf{i} = 0$, and $\mathbf{j} = \mathbf{n} \times \mathbf{i}$. Any unit vector parallel to the polygon can be described as $\mathbf{v}(\varphi) = \mathbf{i} \cos \varphi + \mathbf{j} \sin \varphi$. The problem of finding the principal curvature at $\mathbf{x}$ thus becomes an 1D optimization problem: finding φ that maximizes $|\kappa_{\text{est}}(\mathbf{x}, \mathbf{v}(\varphi))|$. A conventional numerical optimization

scheme such as bi-section search, Newton's method, or Powell's method, can solve this problem.

5.5 Limitation of the proposed curvature estimation scheme

The proposed curvature estimation scheme calculates the curvature accurately over a model that has clear distinction between a curved region and a noncurved region, such as a CAD-generated model of a mechanical part. However, the estimated curvature can have some error from the actual curvature. The error becomes relatively large in a region where the curvature changes gradually. Since the program cannot judge if such a gradual curvature change is correct or an artifact created by an inappropriate tessellation, the program cannot calibrate the curvature estimation for such a gradual curvature change. Also, when the crawling path is cut short by a geometric feature, the estimated curvature tends to be larger than the actual curvature because the length of the crawling path, which is the denominator of the estimated curvature, is shorter. When the error of the curvature estimation is large, some of the feature edges may be overlooked, or some edges are falsely identified as feature edges. Nonetheless, the proposed method is sufficiently capable of capturing a change of curvature and thus effectively detects type 2 edges shown in Fig. 3e.

6 Extracting feature edges

The proposed method extracts three types of feature edges in the following order:

1. feature edges on sharp corners,
2. feature edges across which the curvature changes suddenly,
3. feature edges located on the cylindrical surface.

For finding types 1 and 2 feature edges, the proposed method tests each edge. For type 3 feature edges, the proposed method takes one sample point per polygon and tests whether the polygon is a part of a cylindrical surface.

The total computational time for finding feature edges is thus roughly proportional to the number of edges and the number of polygons.

The following section explains the methods for extracting feature edges of the three types.

6.1 Extracting feature edges on sharp corners

The proposed method first extracts feature edges on sharp corners. The proposed method tests each edge of the polygonal surface shared by two polygons; if the normal vectors of the two polygons sharing the edge make a larger angle than a given angle threshold ω_{sharp} , the edge is extracted as a feature edge.

6.2 Extracting feature edges based on sudden curvature change

The proposed method extracts feature edges by finding a sudden change of estimated curvature across an edge. For this purpose, the curvature is measured in the direction perpendicular to the edge at two points, one point each on a polygon sharing the edge.

The proposed method visits each edge shared by two polygons and estimates the curvature for each polygon sharing the edge as follows: Let $\mathbf{e}_1$ and $\mathbf{e}_2$ be the vertices of the edge being tested, and $\mathbf{n}$ is the normal of the polygon. The proposed method shoots two rays from $(\mathbf{e}_1 + \mathbf{e}_2)/2$ to one of the directions $(\mathbf{e}_2 - \mathbf{e}_1) \times \mathbf{n}$ or $(\mathbf{e}_1 - \mathbf{e}_2) \times \mathbf{n}$ whichever points inside the polygon and finds the first intersection $\mathbf{e}_x$. A point inside the polygon $\mathbf{x}_{\text{mid}}$ is then found as:

$$\mathbf{x}_{\text{mid}} = (\mathbf{e}_x + (\mathbf{e}_1 + \mathbf{e}_2)/2)/2,$$

as shown in Fig. 9. Since the edge is used by two polygons, two of such points, $\mathbf{x}_{\text{mid}1}$ and $\mathbf{x}_{\text{mid}2}$, are found. The proposed method then estimates curvatures at $\mathbf{x}_{\text{mid}1}$ and $\mathbf{x}_{\text{mid}2}$ as:

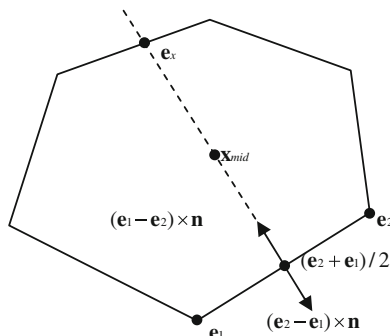


Fig. 9 Finding a point inside a polygon

$$\kappa_{\text{est}1} = \kappa_{\text{est}}(\mathbf{x}_{\text{mid}1}, \mathbf{v}_{\text{mid}1}), \text{ and}$$

$$\kappa_{\text{est}2} = \kappa_{\text{est}}(\mathbf{x}_{\text{mid}2}, \mathbf{v}_{\text{mid}2}),$$

where $\mathbf{v}_{\text{mid}K}$ is a unit vector parallel to $(\mathbf{x}_{\text{mid}K} - (\mathbf{e}_1 + \mathbf{e}_2)/2)$, $\{K = 1, 2\}$. An example of the curvature estimation at two points on the both sides of an edge is shown in Fig. 10.

Finally, if one of the following two conditions is met, the edge is extracted as a feature edge.

- (1) $\kappa_{\text{est}1}$ and $\kappa_{\text{est}2}$ are reliable with respect to the conditions described in Sect. 5.3, and the ratio of $|\kappa_{\text{est}1}|$ and $|\kappa_{\text{est}2}|$ is greater than a given threshold γ_1 (i.e., $|\kappa_{\text{est}1}| > \gamma_1 |\kappa_{\text{est}2}|$ or $|\kappa_{\text{est}2}| > \gamma_1 |\kappa_{\text{est}1}|$)
- (2) Only one of $\kappa_{\text{est}1}$ and $\kappa_{\text{est}2}$ is reliable with respect to the conditions explained in Sect. 5.3, and the ratio of $|\kappa_{\text{est}1}|$ and $|\kappa_{\text{est}2}|$ is greater than a given threshold γ_2 (i.e., $|\kappa_{\text{est}1}| > \gamma_2 |\kappa_{\text{est}2}|$ or $|\kappa_{\text{est}2}| > \gamma_2 |\kappa_{\text{est}1}|$)

The threshold γ_1 describes how much change of curvature across an edge is required for the edge to be extracted as a feature edge. The edge is extracted as a feature edge if the curvature changes γ_1 times across the edge. Hence, γ_1 must be chosen based on the user's requirement.

The choice of γ_2 must depend on the percentage of the error caused by the unreliability described in Sect. 5.3. For example, if the estimated curvature κ_{est} is subject to an error ratio of E ($E > 1$), κ_{est} can vary in the range of $\kappa/E \leq \kappa_{\text{est}} \leq \kappa E$, where κ is the exact curvature. To take the error into account, γ_2 must be E_{γ_1} . However, because the percentage of error E cannot be obtained, γ_2 can only be found empirically.

6.3 Detecting feature edges on the cylindrical surfaces

The third procedure in the proposed method extracts feature edges on the cylindrical surfaces that are parallel to the axis of the cylinder. It is desirable to constrain four to six edges on the cylindrical surface that are parallel to the axis of the cylinder for the meshing; such constraints become particularly important when a coarse mesh is required. As can be seen in Fig. 3c, d, the shape of the cylinder is not preserved unless some edges parallel to the axis are constrained. However, such feature edges cannot be captured by the sudden change of the curvature because the curvature is uniform on a cylindrical surface.

It is also important to constrain edges on the co-axial cylindrical surfaces consistently so that meshing does not produce self-intersections. For example, if two co-axial cylindrical surfaces have constrained edges as shown in Fig. 11 (thick lines are constrained edges), meshing produces self-intersections. Such self-intersections can be avoided by creating sufficiently small elements on the cylindrical surfaces. However, large elements often need to be created on such cylindrical surfaces due to the limitation

Fig. 10 Example of curvature estimation on both sides of an edge. **a** Crawling path originated from $\mathbf{x}_{mid1}$, **b** crawling path originated from $\mathbf{x}_{mid2}$

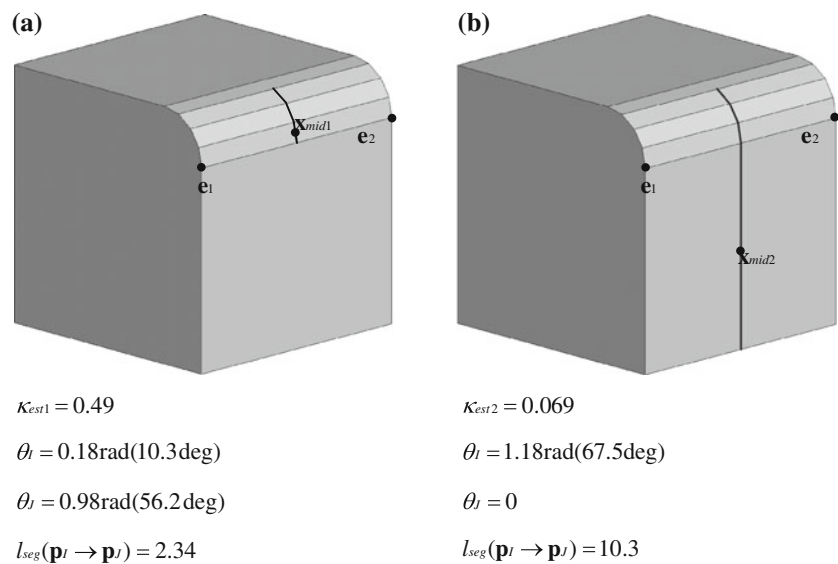
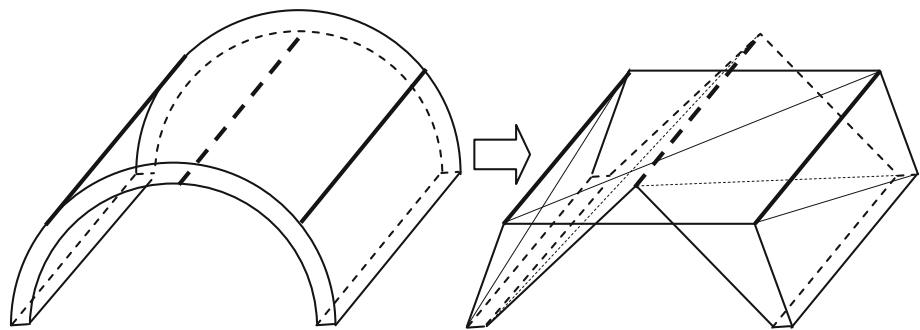


Fig. 11 Inappropriately constrained edges yield self-intersection after meshing



of the number of elements. Hence, the proposed method needs to find cylindrical surfaces from the polygonal surface, and extract feature edges on them consistently so that the chance of yielding such self-intersections is minimized.

If some polygons represent a complete cylinder (a cylinder whose cross-section is a complete circle), the cylinder can be found by fitting a circle to a crawling path in the principal curvature direction and checking the fitness of the circle to the crawling path. However, it is difficult to identify some polygons representing a partial cylinder (a cylinder whose cross-section makes a partial circle) because such polygons can also be interpreted as a part of a noncylindrical surface.

For this reason, a heuristic approach is used for finding partial cylinders. Often a complete cylinder is co-axial to a partial cylinder; when the proposed method finds some polygons that potentially represent a partial cylinder, those polygons are considered to be a part of a cylindrical surface if a complete cylinder co-axial to the partial cylinder is located close to the partial cylinder.

The proposed method uses two angle thresholds, ω_{cy11} and ω_{cy12} ($\omega_{cy11} < \omega_{cy12}$), to find cylindrical surfaces. When the proposed method locates a set of polygons that may represent

a cylindrical surface, the proposed method calculates the angle of arc of the cross-section of the potential cylindrical surface, denoted as θ_{cy1} . The surface represented by the polygons is considered to be a complete cylinder if $\omega_{cy12} \leq \theta_{cy1}$. If $\omega_{cy11} \leq \theta_{cy1} < \omega_{cy12}$, the surface represented by the polygons is considered to be a partial cylinder if a co-axial complete cylinder is found. The surface represented by the polygons is not considered to be a part of a cylindrical surface if $\theta_{cy1} < \omega_{cy11}$. Since θ_{cy1} is calculated by taking an angle of revolution made by a crawling path around the surface, we call θ_{cy1} a “revolution angle” of the cylinder.

After locating cylindrical surfaces represented by sets of polygons the proposed method makes groups of cylindrical surfaces based on the proximity and axial directions of the cylinders. A coordinate frame is assigned to each group, and feature edges are extracted on the cylindrical surfaces based on the coordinate frame. Details of the algorithm are described below.

The proposed method calculates a principal curvature κ_1 and the principal curvature direction $\mathbf{v}_{\kappa1}$ at $\mathbf{x}$ by the method described in Sect. 5.4 where $\mathbf{x}$ is a point on the polygonal surface. The method then finds two crawling paths starting from $\mathbf{x}$ to the directions $\mathbf{v}_{\kappa1}$ and $-\mathbf{v}_{\kappa1}$ until:

- (1) total angular change of the average tangential vector exceeds 180° ,
- (2) angular change of the normal vectors of polygons in one step of crawling exceeds ω_{sharp} , or
- (3) twist angle exceeds ω_{twist} .

The two crawling paths described as $\{\mathbf{a}_0, \mathbf{a}_1, \dots, \mathbf{a}_N\}$ and $\{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_M\}$ are merged into a crawling path:

$$C_{\text{cyl}} = \{\mathbf{p}_{-M-1}, \mathbf{p}_{-M}, \dots, \mathbf{p}_N\} \\ = \{\mathbf{b}_M, \mathbf{b}_{M-1}, \dots, \mathbf{b}_0, \mathbf{a}_0, \mathbf{a}_1, \dots, \mathbf{a}_N\}.$$

The revolution angle θ_{cyl} is calculated from C_{cyl} as:

$$\theta_{\text{cyl}} = \theta_{\text{seg}}(\mathbf{p}_{-M-1} \rightarrow \mathbf{p}_N),$$

which is the total angular change of the tangential vector of the crawling path from $\mathbf{p}_{-M-1}$ to $\mathbf{p}_N$. If the crawling path is on a complete cylindrical surface and makes a complete revolution around the cylinder, θ_{cyl} is 360° . If the crawling path is on a partial cylindrical surface, θ_{cyl} is less than 360° .

If the crawling path is on a cylindrical surface, the plot of normal vectors lies on a plane intersecting with the origin as shown in Fig. 12. Hence, the axial direction of the cylinder can be obtained by fitting a plane to the normal vector plot and taking the normal of the plane. The proposed method fits a plane to the normal vectors of the polygons that the crawling path passes by the least-square method. The plane found by fitting to the normal vectors is denoted as P_n , and the axial direction is denoted as $\mathbf{a}$.

The points of the crawling path are then projected on P_n , and a circle that fits to the projected points is calculated by the least-square method [10, 11]. The center of the circle is $\mathbf{o}$, and the diameter is R . To test the fitness of the circle to the projected points, the proposed method calculates the maximum and the minimum distances from a projected point to the center of the circle, R_{max} and R_{min} .

Finally, the proposed method considers that the polygons passed by the crawling path are part of a cylindrical surface if the following three conditions are satisfied: (1)

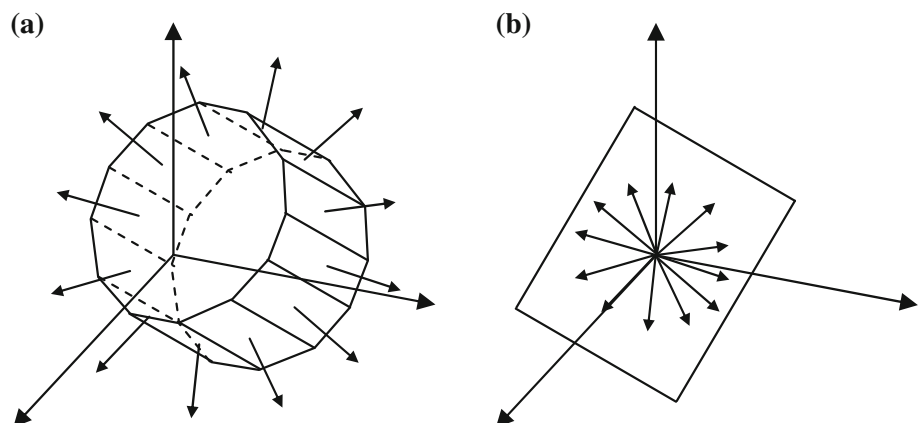
$$\omega_{\text{cyl1}} \leq \theta_{\text{cyl}}, \quad (2) \quad 0.95 \leq R_{\text{max}}/R \leq 1.05, \quad \text{and} \quad (3) \quad 0.95 \leq R_{\text{min}}/R \leq 1.05.$$

The proposed method tests the above conditions at a sample point per polygon and makes a list of detected cylindrical surfaces. If the polygon is convex, the sample point for the polygon is the center of gravity of the polygon. If the polygon is concave, the proposed method searches randomly a point inside the polygon and uses it as a sample point for the polygon.

Groups of cylindrical surfaces are then made by the following algorithm. Let $C_{\text{cyl},1}, C_{\text{cyl},2}, \dots, C_{\text{cyl},N}$ be the detected cylinders sorted in the descending order of the revolution angle, and $\theta_{\text{cyl},k}$ be the revolution angle of $C_{\text{cyl},k}$ ($1 \leq k \leq N$); $\theta_{\text{cyl},i} \geq \theta_{\text{cyl},j}$ ($i < j$). Assume that $C_{\text{cyl},I}$ is the first cylinder that is not yet in any group and satisfies $\omega_{\text{cyl2}} \leq \theta_{\text{cyl},I}$. The proposed method makes a new group of cylindrical surfaces, $G_{\text{cyl},K}$ (K th group), that initially includes only $C_{\text{cyl},I}$. The proposed method then compares $C_{\text{cyl},I}$ with $C_{\text{cyl},J}$ where $I+1 \leq J \leq N$, and $C_{\text{cyl},J}$ is added to $G_{\text{cyl},K}$ if (1) the axial directions of $C_{\text{cyl},I}$ and $C_{\text{cyl},J}$ are not different by more than angle ω_{shallow} , and (2) cylindrical surfaces $C_{\text{cyl},I}$ and $C_{\text{cyl},J}$ intersect each other. The two conditions are for checking if $C_{\text{cyl},I}$ and $C_{\text{cyl},J}$ are co-axial to each other. The two conditions, however, can be satisfied if the axis of one cylinder is off center of the other. The two conditions are intentionally made loose for tolerating discretization errors. The proposed method repeats this process until all the cylinders that satisfy $\omega_{\text{cyl2}} \leq \theta_{\text{cyl},I}$ are included in a group. Some cylinders with less than ω_{cyl2} revolution angle may not belong to any group, and feature edge will not be extracted on such cylinders.

Finally, the proposed method calculates a coordinate frame for each group and makes feature edges on the side surfaces of the cylinders included in the group. $\mathbf{a}_K$ is a unit vector that is parallel to the axis of the first cylinder of the group $G_{\text{cyl},K}$, $\mathbf{i}$ is an arbitrary unit vector that is perpendicular to $\mathbf{a}_K$, and $\mathbf{j}$ is $\mathbf{i} \times \mathbf{a}_K$. The coordinate frame is defined by the three vectors $\mathbf{i}$, $\mathbf{j}$, and $\mathbf{a}_K$. The proposed

Fig. 12 Normal vector plot of a discretized cylindrical surface.
a A discretized cylindrical surface and normal vectors,
b a plot of normal vectors lying on a plane intersecting with the origin



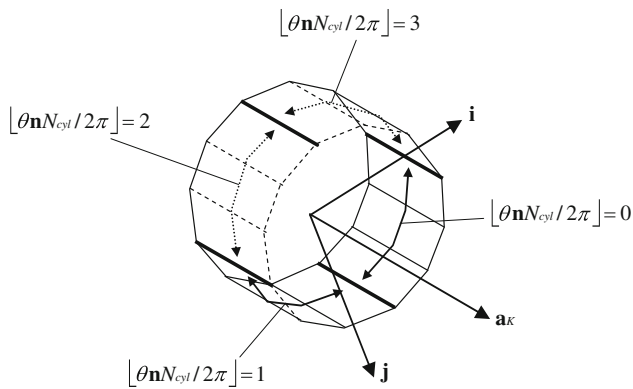


Fig. 13 Feature edges extracted around a cylinder (thick lines are extracted feature edges)

method assigns to each polygon included in the cylinders of the group, a clockwise rotation angle measured from $\mathbf{i}$ to the normal of the polygon $\mathbf{n}$ ($\theta_{\mathbf{n}} = \arccos(\mathbf{i} \cdot \mathbf{n})$ if $\mathbf{j} \cdot \mathbf{n} \geq 0$, $\theta_{\mathbf{n}} = 2\pi - \arccos(\mathbf{i} \cdot \mathbf{n})$ otherwise.) The proposed method extracts as a feature edge an edge shared by two polygons included in a cylinder of the group that satisfies $\lfloor \theta_{\mathbf{n}_1} N_{\text{cyl}} / 2\pi \rfloor \neq \lfloor \theta_{\mathbf{n}_2} N_{\text{cyl}} / 2\pi \rfloor$, where $\theta_{\mathbf{n}_1}$ and $\theta_{\mathbf{n}_2}$ are the angles assigned to the two polygons sharing the edge, N_{cyl} is a user-specified threshold that defines the number of feature edges needed around a complete cylinder, and $\lfloor x \rfloor$ is the largest integer that is less than x . For example, when

Table 2 Parameters used for creating examples

ω_{shallow}	5°	γ_2	10
ω_{sharp}	45°	ω_{cyl1}	45°
ω_{twist}	45°	ω_{cyl2}	270°
γ_1	2	N_{cyl}	6

N_{cyl} is four, $\lfloor \theta_{\mathbf{n}} N_{\text{cyl}} / 2\pi \rfloor$ varies zero to three, and a feature edge is extracted where $\lfloor \theta_{\mathbf{n}} N_{\text{cyl}} / 2\pi \rfloor$ changes as shown in Fig. 13.

In most cases, a CAD-generated polygonal surface does not include an excessively large number of cylindrical surfaces. However, when the input polygonal surface includes a large number of cylindrical surfaces, grouping of detected cylindrical surface may take longer. In the worst case, the computational time for the grouping becomes the order of N^2 , where N is the number of detected cylindrical surfaces. However, the grouping process can be accelerated by making preliminary grouping based on the axial direction. The preliminary grouping is based on the grouping of unit vectors, and it is equivalent to grouping of points scattered over a spherical surface. The computation cost for the preliminary grouping can be easily reduced to the order of N , where N is the number of detected cylindrical surfaces. The final grouping of the cylindrical surfaces can be

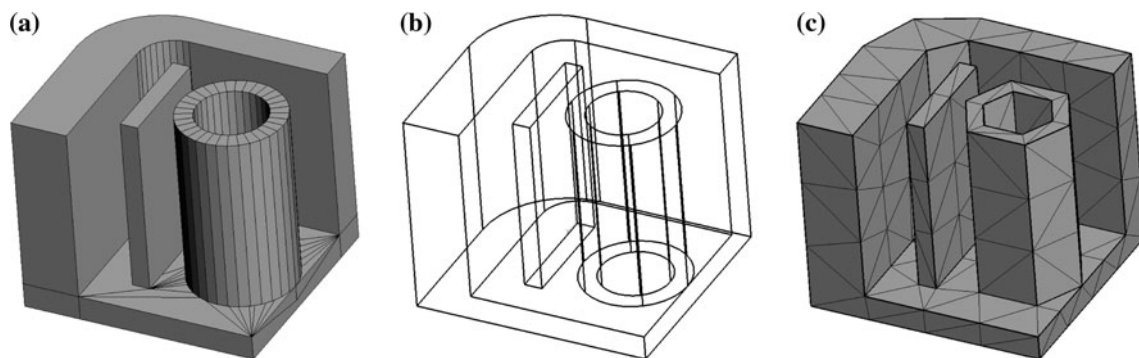


Fig. 14 A mechanical part. **a** Input model (235 vertices and 200 polygons), **b** extracted feature edges (<0.1 s with AMD Athlon 64, 2.2 GHz), **c** a coarse mesh with feature edges constrained (165 vertices and 326 triangles)

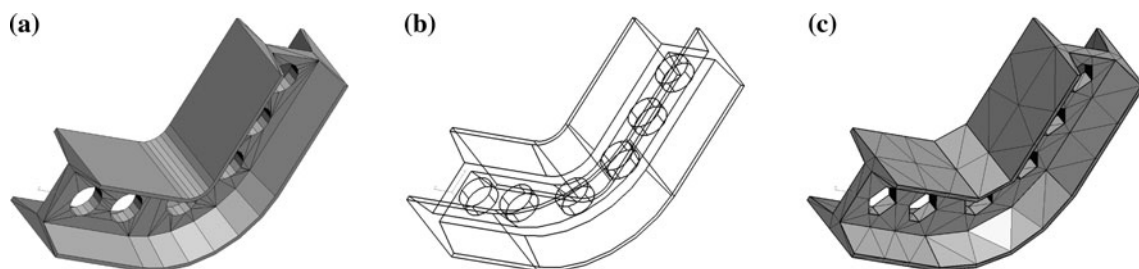


Fig. 15 A bent beam with holes. **a** Input model (356 vertices and 348 polygons), **b** extracted feature edges (<0.1 s with AMD Athlon 64, 2.2 GHz), **c** a coarse mesh with feature edges constrained (204 vertices and 428 triangles)

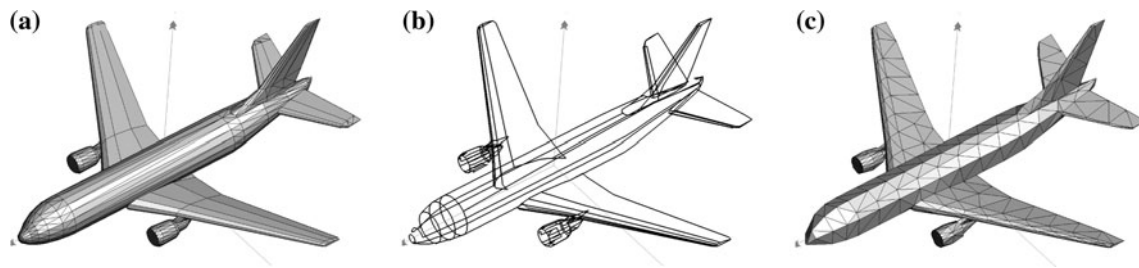


Fig. 16 B767. **a** Input model (1,661 vertices and 1,082 polygons), **b** extracted feature edges (1.3 s with AMD Athlon 64, 2.2 GHz), **c** a mesh with feature edges constrained (444 vertices and 884 triangles)

performed within each preliminary group. Since each preliminary group is expected to have only few cylindrical surfaces, the computational cost for the grouping within a preliminary group is usually small.

7 Results

The proposed method has been tested with many polygonal surfaces including more than 50 facet models used for

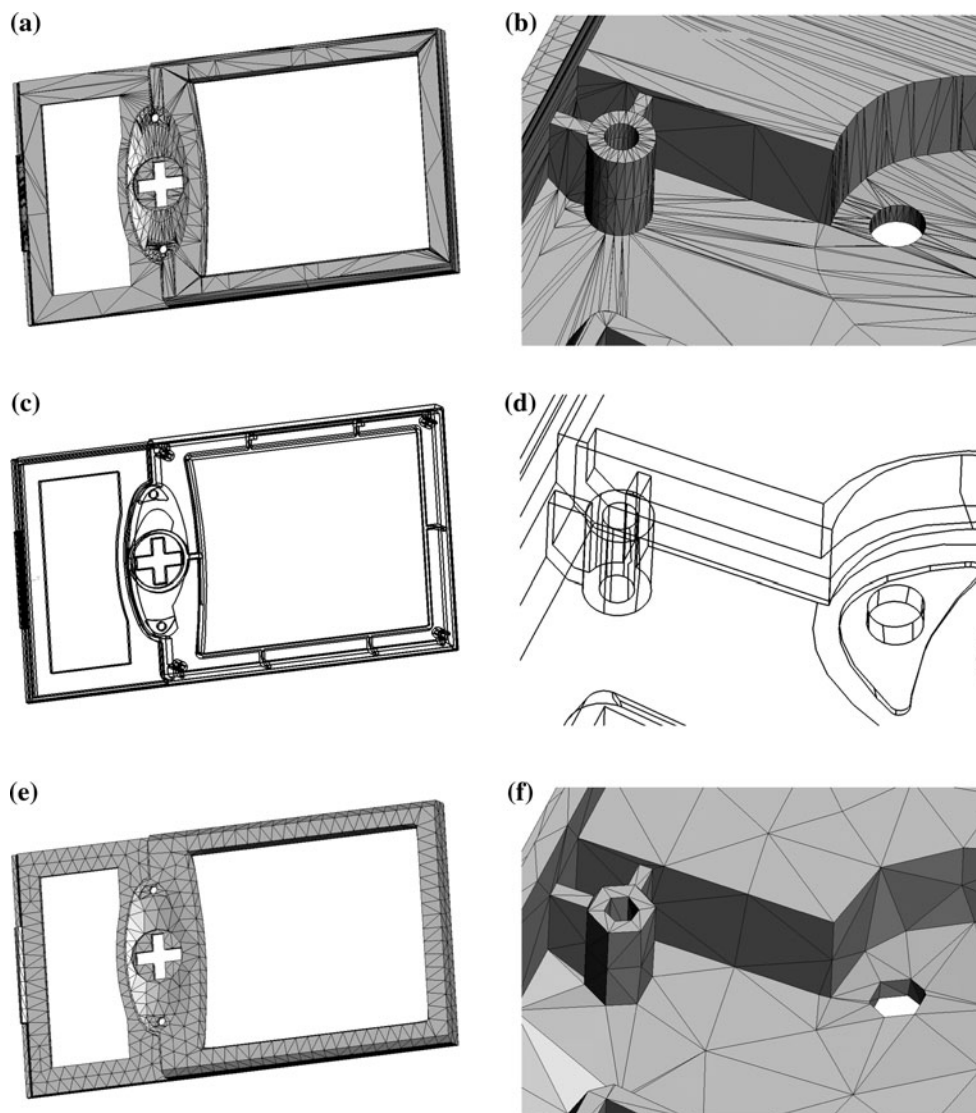
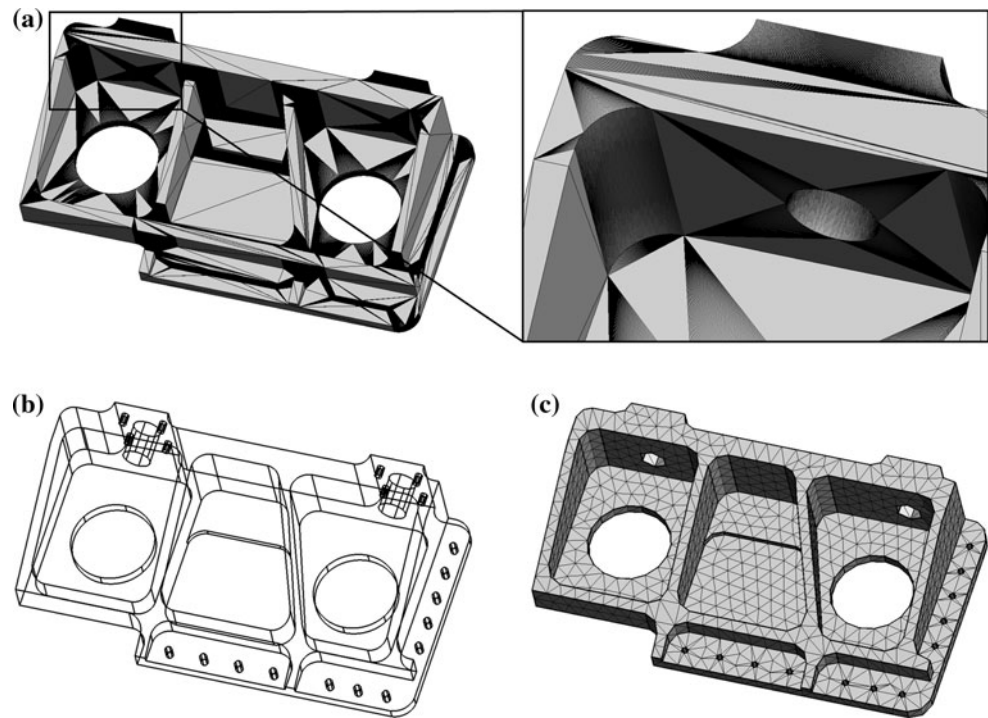


Fig. 17 A front cover of PDA. **a** Input model (5,336 vertices and 10,692 polygons), **b** close-up of holes in the input model, **c** extracted feature edges (5.4 s with AMD Athlon 64, 2.2 GHz), **d** extracted

feature edges near the holes, **e** a triangular mesh (2,007 vertices and 4,034 polygons), **f** close-up of the triangular mesh near the holes

Fig. 18 A mechanical part. **a** Input model (9,945 vertices, 19,950 polygons), **b** extracted feature edges, **c** triangular mesh (1,763 vertices, 3,586 polygons)



plastic injection molding simulation, and has shown good performance. This section shows some results from the experiments. The parameters used for creating these models are shown in Table 2.

Figures 14 and 15 demonstrate the ability of the proposed method in capturing sharp corners, fillet boundaries, and cylindrical surfaces. Both models include concave polygons with more than four nodes. The proposed method successfully captures the cylindrical surfaces and extracted six feature edges on each cylindrical surface. Figure 14b shows that the proposed method extracts feature edges on the two co-axial cylinders so that the feature edges on one of the two cylinders is located close to a feature edge on the other cylinder to reduce the possibility of producing self-intersections when large elements must be created on the cylinders. Both models are successfully meshed into a very coarse mesh without producing self-intersections. Creating a very coarse mesh without producing self-intersections is very difficult. It is only possible when appropriate edges are constrained.

Figure 16 shows a more complex example. The input model includes concave polygons with more than four vertices. The proposed method successfully extracts feature edges from the input model, and the model is meshed into a coarse triangular mesh.

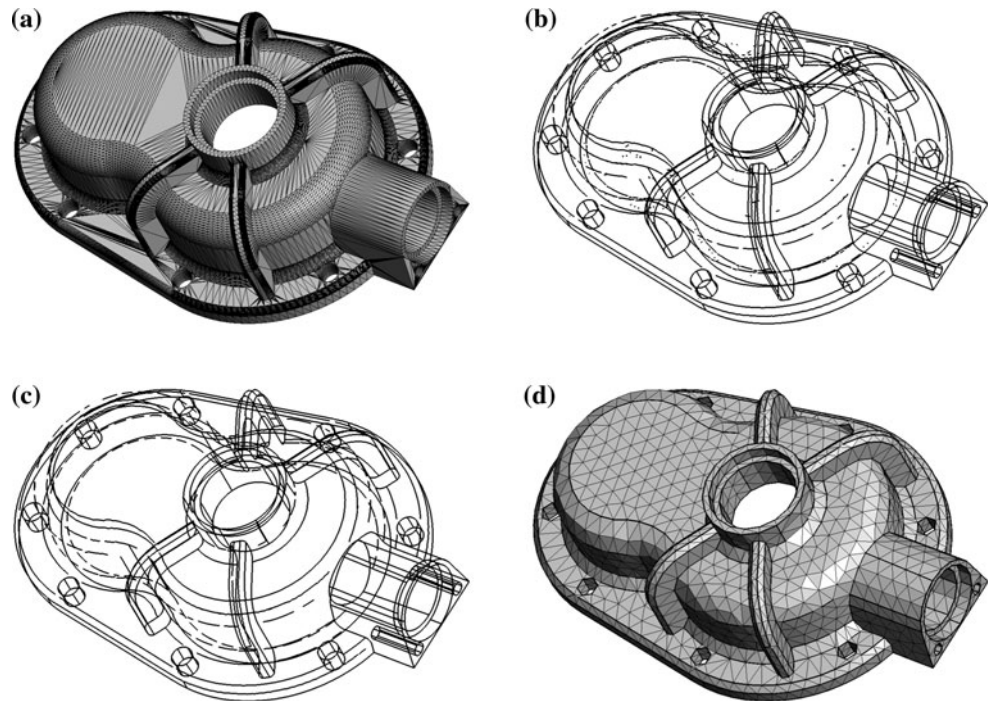
Figure 17 shows an example of a PDA cover. This model is created by a CAD facet generator and includes many high aspect-ratio triangles. The model has some small cylindrical holes as shown in Fig. 17b; such holes

create self-intersections when a coarse mesh is needed unless feature edges are constrained appropriately. The proposed method successfully extracts feature edges on the holes, and a valid triangular mesh is shown in Fig. 17e, f.

Figure 18 shows an example of a mechanical part. This model is created by a CAD facet generator with a small error tolerance. As can be seen from Fig. 18a, curved areas are tessellated into a set of dense triangles. The triangle density, however, is small in low-curvature directions. The proposed method extracts all the fillet boundaries and edges along cylindrical holes, as shown in Fig. 18b. The model is successfully meshed as shown in Fig. 18c.

Figure 19 shows another example of a mechanical part. Again, this model is created by a CAD facet generator with a small error tolerance. Due to the small error tolerance, the triangle density over the curved area is high while fewest-possible triangles are placed over a flat region. Although the error from the polygonal surface to the original surface is small, it includes some unevenness due to poor tessellation. It is not possible to completely eliminate effects of such unevenness, and thus some small pieces of feature edges are created as shown in Fig. 19b. In this example, such small pieces are successfully removed by excluding feature edges shorter than 0.1 mm as shown in Fig. 19c, and a triangular mesh is successfully created as shown in Fig. 19d. Nonetheless, some discontinuous pieces of feature edges remain after removing small pieces, and it would be ideal if some of them were either connected together or deleted. Such deletion or connection could be achieved by a sophisticated

Fig. 19 A mechanical part.
a Input model (21,034 vertices and 42,104 polygons),
b extracted feature edges,
c extracted feature edges 0.1 mm or shorter feature edges excluded,
d triangular mesh (2,284 vertices and 4,604 polygons)



post-processing algorithm for extracted feature edges. This is one of our future research topics.

8 Discussion and conclusions

The proposed method extracts feature edges from a general polygonal surface. The method extracts (1) edges on sharp corners, (2) edges where the curvature changes significantly across the edge, and (3) edges on the cylindrical surfaces. Such feature edges must be constrained for meshing, or some geometric features will be lost unless sufficiently small elements are used. The method has been tested with many polygonal surfaces and has shown a good performance on the test models.

The method helps to create a coarse mesh without producing self-intersections. However, the method does not necessarily help to create good quality elements. If two feature edges are parallel and close to each other, elements created between the two feature edges will show very high aspect ratio. If such elements must be avoided, the proposed method must be used together with an appropriate element sizing function. Therefore, the proposed method does not eliminate the necessity of the element sizing function.

The method captures cylindrical surfaces and extracts edges on such cylindrical surfaces. However, the method does not capture when the cross-section is an ellipse, and/or the surface is significantly tapered. Capturing such elliptic cylinders and conic surfaces is a topic for future research.

References

1. Mangan AP, Whitaker RT (1999) Partitioning 3D surface meshes using watershed segmentation. *IEEE Trans Vis Comput Graph* 5:308–321
2. Nomura M, Hamada N (2001) Feature edge extraction from 3D triangular meshes using a thinning algorithm. In: *Proceedings of SPIE-vision geometry X*, pp 34–41
3. Jiao X, Heath MT (2002) Feature detection for surface meshes. In: *Proceedings of 8th international conference on numerical grid generation in computational field simulations*, pp 705–714
4. Jiao X, Heath MT (2004) Overlaying surface meshes, part II: topology preservation and feature detection. *Int J Comput Geom Appl* 14:403–419
5. Sun Y, Page DL, Paik JK, Koschan A, Abidi MA (2002) Triangle mesh-based edge detection and its application to surface segmentation and adaptive surface smoothing. In: *Proceedings of international conference on image processing*, pp 825–828
6. Baker TJ (2004) Identification and preservation of surface features. In: *Proceedings of 13th international meshing roundtable*, pp 299–309
7. Gumhold S, Wang X, MacLeod R (2001) Feature extraction from point clouds. In: *Proceedings of 10th international meshing roundtable*, pp 293–305
8. Pauly M, Keiser R, Gross M (2003) Multi-scale feature extraction on point-sampled surfaces. *Comput Graph Forum* 22(3):281–289
9. Jiao X (2006) Volume and feature preservation in surface mesh optimization. In: *Proceedings of 15th international meshing roundtable*, pp 359–373
10. Gander W, Golub GH, Strebler R (1994) Least square fitting of circles and ellipses. *BIT* 34:558–578
11. Coope ID (1993) Circle fitting by linear and nonlinear least squares. *J Optim Theory Appl* 76:381–388